

What is HealthCheck in .NET? How we are using it Please explain in detail for experienced interview

Health Checks in .NET Core — Experienced Interview Answer

Interview opening (30 sec):

Health Checks in ASP.NET Core are a built-in monitoring mechanism that exposes the health of an application and its dependencies (SQL Server, Redis, external APIs, queues, etc.) through HTTP endpoints. They're mainly used by Kubernetes, Azure App Service, load balancers, and monitoring tools to decide whether an application is alive, ready to receive traffic, or unhealthy.

1. Why do we need Health Checks?

Imagine your API is running, but SQL Server is down.

Without Health Checks:

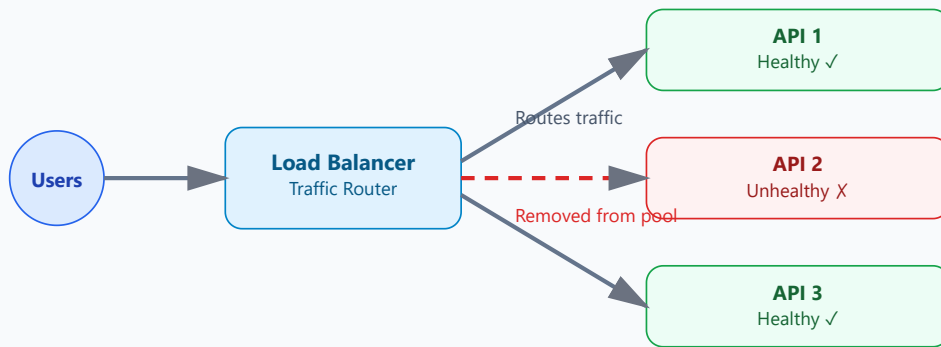
- IIS/Kubernetes still thinks the API is healthy.
- Users receive 500 errors.

With Health Checks:

- The application reports **Unhealthy**.
- Load balancer removes that instance.
- Traffic goes to healthy servers.

Real Production Architecture

Health Checks in Production

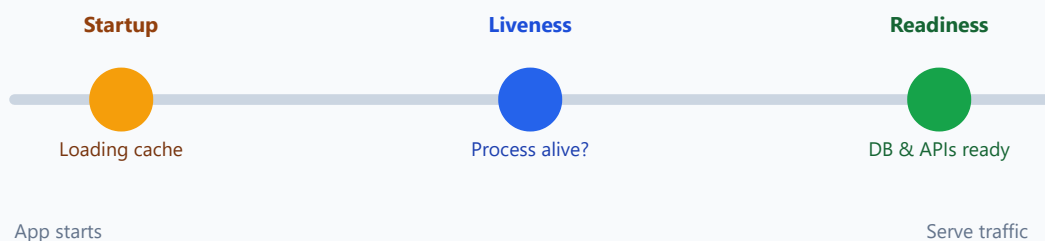


2. Health Check Types (Very Important)

Type	Purpose	Example
Liveness	Is app alive?	Process running
Readiness	Can app serve requests?	DB connected
Startup	Has app finished starting?	Cache loaded

Easy Interview Explanation

Health Check Timeline



Liveness

Checks only whether the application process is alive.

`</> http`



If this fails → restart the container.

Readiness

Checks whether dependencies are available.

```
</> http
```



If SQL fails → remove instance from load balancer.

Startup

Used for long startup applications.

```
</> http
```



3. How to Implement Health Checks

Step 1: Register Services

```
</> C#
```



```
builder.Services.AddHealthChecks()  
    .AddSqlServer(  
        builder.Configuration.GetConnectionString("Default"),  
        name: "SQL Server")  
    .AddRedis(  
        builder.Configuration["Redis"],  
        name: "Redis");
```

What's happening?

- SQL connectivity
- Redis availability
- Returns Healthy / Unhealthy automatically

Step 2: Map Endpoint

```
</> C#
```



```
app.MapHealthChecks("/health");
```

Now access:

```
https://api.company.com/health
```



Response:

```
</> JSON
```



```
Healthy
```



4. Custom Health Check (Most Asked)

Suppose your application depends on an external Payment API.

Create Custom Health Check

```
</> C#
```



```
public class PaymentApiHealthCheck : IHealthCheck
{
    private readonly HttpClient _client;

    public PaymentApiHealthCheck(HttpClient client)
    {
        _client = client;
    }

    public async Task<HealthCheckResult> CheckHealthAsync(
        HealthCheckContext context,
        CancellationToken token)
    {
        var response = await _client.GetAsync("/status");

        return response.IsSuccessStatusCode
            ? HealthCheckResult.Healthy()
            : HealthCheckResult.Unhealthy("Payment API unavailable");
    }
}
```

Register:

</> C#



```
builder.Services.AddHealthChecks()  
    .AddCheck<PaymentApiHealthCheck>("Payment API");
```

5. Health Status Levels

Status	Meaning
Healthy	Everything working
Degraded	App works, but partially affected
Unhealthy	Cannot serve requests

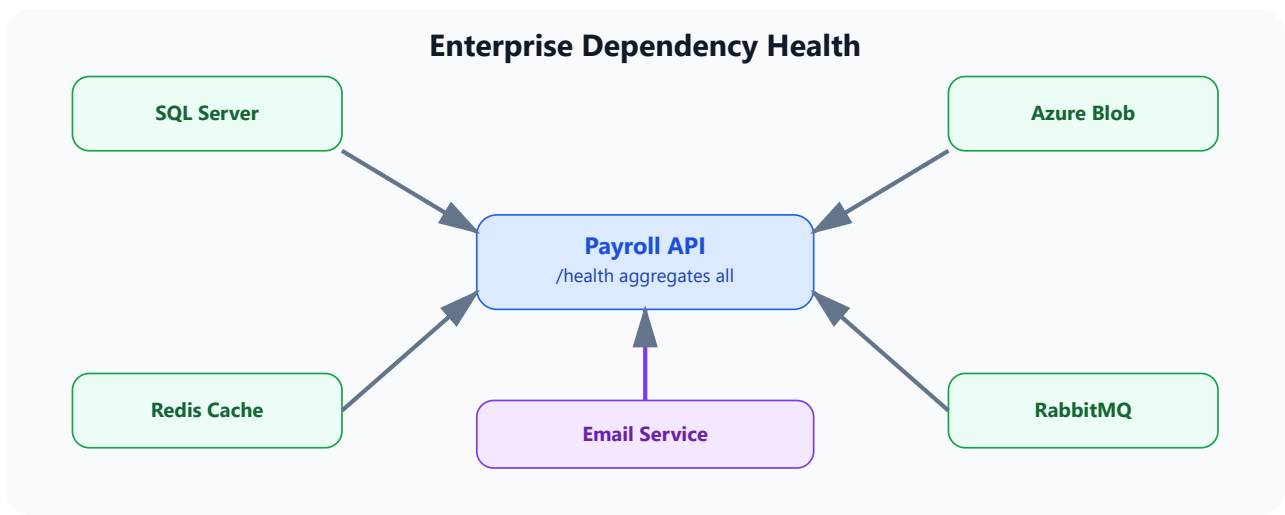
Example



6. Real Enterprise Example

Suppose you're working on Payroll System.

Dependencies



Health endpoint checks all these.

If RabbitMQ fails:

</> JSON

```
{
  "status": "Degraded",
  "entries": {
    "SQL": "Healthy",
    "Redis": "Healthy",
    "RabbitMQ": "Unhealthy"
  }
}
```

Application still works → mark **Degraded** instead of Unhealthy.

7. Separate Endpoints (Best Practice)

</> C#

```
builder.Services.AddHealthChecks()
    .AddSqlServer(connectionString, tags: new[] { "ready" })
    .AddRedis(redis, tags: new[] { "ready" });

app.MapHealthChecks("/health/live",
    new HealthCheckOptions
    {
        Predicate = _ => false
    });

app.MapHealthChecks("/health/ready",
    new HealthCheckOptions
    {
```

```
Predicate = check => check.Tags.Contains("ready")
});
```

Endpoint	Used By
/health/live	Kubernetes restart
/health/ready	Load Balancer
/health	Monitoring dashboard

8. Health Check Response Customization

Default response is simple.

Experienced teams return JSON.

```
</> C#

app.MapHealthChecks("/health", new HealthCheckOptions
{
    ResponseWriter = async (context, report) =>
    {
        context.Response.ContentType = "application/json";

        await context.Response.WriteAsJsonAsync(new
        {
            Status = report.Status.ToString(),
            Duration = report.TotalDuration,
            Checks = report.Entries.Select(x => new
            {
                Name = x.Key,
                Status = x.Value.Status.ToString(),
                Description = x.Value.Description
            })
        });
    }
});
```

Response:

```
</> JSON

{
  "status": "Healthy",
  "duration": "00:00:00.45",
  "checks": [
```

```
{
  "name": "SQL Server",
  "status": "Healthy"
}
]
```

9. Health Checks vs Exception Handling

Health Checks	Exception Handling
Proactive monitoring	Reactive handling
Checks dependencies	Handles runtime errors
Used by DevOps	Used by application
Returns system status	Returns API error

Interview line:

Global exception handling protects users. Health checks protect infrastructure.

10. Kubernetes Integration

Kubernetes YAML:

```
</>  YAML

livenessProbe:
  httpGet:
    path: /health/live
    port: 80

readinessProbe:
  httpGet:
    path: /health/ready
    port: 80
```



Behavior:

Result	Kubernetes Action
Liveness fails	Restart Pod

Readiness fails	Stop routing traffic
Healthy	Continue serving

11. Scenario-Based Interview Questions

Scenario 1

Interviewer: Your API is running, but SQL Server is down. What happens?

Answer:

- Liveness → Healthy (process alive)
- Readiness → Unhealthy
- Load balancer removes the instance
- Users are redirected to another healthy instance

Scenario 2

Interviewer: Should every dependency make the application unhealthy?

Answer:

No.

Dependency	Status
SQL Server	Unhealthy
Authentication	Unhealthy
Redis Cache	Degraded
Email Service	Degraded
Analytics API	Degraded

Critical services → Unhealthy. Optional services → Degraded.

Scenario 3

Interviewer: Why not call the database from every request instead of using Health Checks?

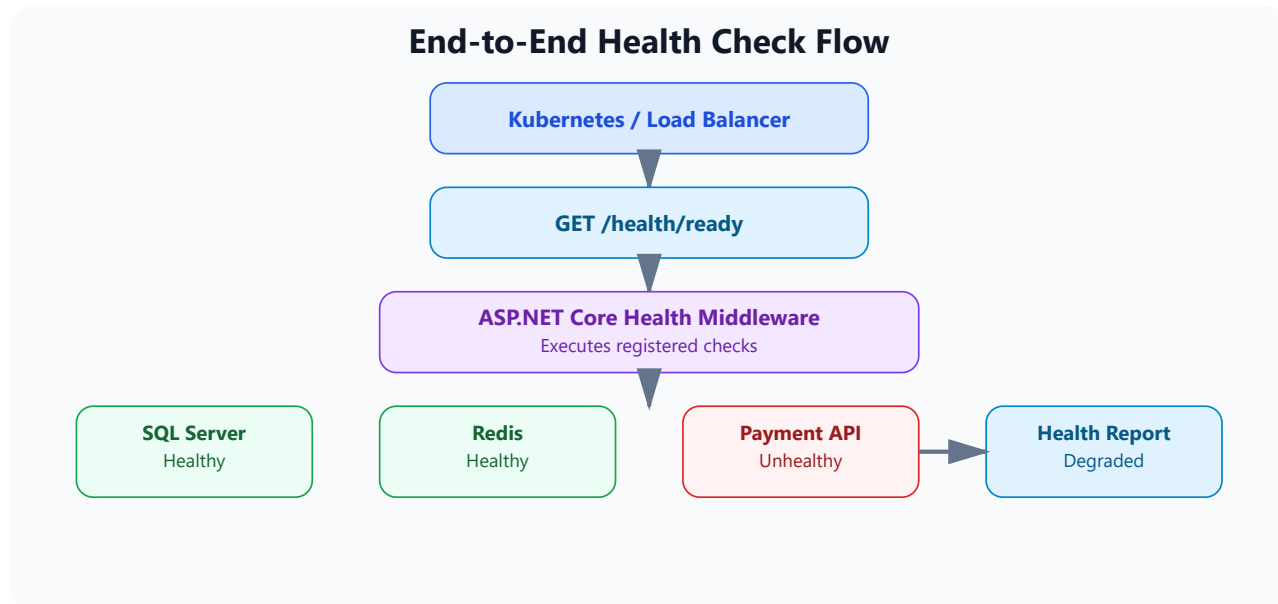
Answer:

Because Health Checks are executed periodically by infrastructure, not by end users. They isolate monitoring from business requests and prevent unnecessary overhead.

12. Production Best Practices

Best Practice	Reason
Separate Live & Ready endpoints	Different infrastructure purposes
Keep health checks lightweight	Avoid slow responses
Use timeouts	Prevent hanging checks
Don't expose sensitive details publicly	Security
Return structured JSON	Better monitoring
Tag health checks	Flexible endpoint filtering

13. Complete Flow (Interview Summary)



15-second closing answer

In my projects, I expose separate `/health/live` and `/health/ready` endpoints. Liveness is used by Kubernetes to restart failed containers, while readiness validates SQL Server, Redis, and external APIs before allowing traffic. I also implement custom `IHealthCheck` classes for business-critical dependencies and return structured JSON so Azure Monitor and Prometheus can track the health of each component individually.

This demonstrates both **development** and **DevOps/production** experience, which interviewers expect from senior .NET developers.

< 2/2 >      ...  Convert to PDF

ChatGPT can make mistakes. Check important info.